

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к лабораторной работе №3
на тему:
**«Исследование работы вычислительных алгоритмов на процессоре и
графическом адаптере»**
БГУИР 6–05–0612–02 95

Выполнил студент группы 453504
РОМАНОВ Николай Викторович

(дата, подпись студента)

Проверил ассистент каф. информатики
КАЛИНОВСКАЯ Анастасия Александровна

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Осуществить методом математического выполнения функции согласно варианту задания. Значение аргумента x изменяется от a до b с шагом h . Для каждого x найти значения функции $Y(x)$, суммы $S(x)$ и число итераций n , при котором достигается требуемая точность $\varepsilon = |Y(x) - S(x)|$.

$$3. \quad S(x) = \sum_{k=0}^n \frac{\cos(\frac{k\pi}{4})}{k!} x^k, \quad Y(x) = e^{x \cos \frac{\pi}{4}} \cos(x \sin \frac{\pi}{4})$$

Рисунок 1.1 – Заданные функции

2 Напишите программу на любом удобном языке программирования, с использованием CUDA или OpenCL. Реализуйте параллельные вычисления, используя возможности GPU/iGPU, чтобы ускорить выполнение задачи. Убедитесь, что ваша программа может обрабатывать более 5000 итераций для получения достоверных результатов.

3 Результатом будут графики нагрузки на GPU.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания был написан код на языке C++, который вычисляет значение функции $Y(x)$ и $S(x)$. В программе используется вставка OpenCL, благодаря которой мы ускорим процесс для большого количества итераций.

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <iomanip>
#include <chrono>
#include <vector>
#include <thread>
#include <atomic>

using namespace std;

inline double asm_add(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "faddp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_mul(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "fmulp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_factorial(int n) {
    if(n == 0 || n == 1) return 1.0;
    double result = 1.0;

    for(int i = 2; i <= n; ++i) {
        double d_i = (double)i;
        __asm__ __volatile__(
            "fldl %1 \n\t"
```

```

        "fldl %0 \n\t"
        "fmlp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "+m" (result)
        : "m" (d_i)
    );
}
return result;
}

std::atomic<long long> global_completed_iters(0);
std::atomic<bool> start_flag(false);

void worker_task(long long iters_for_this_thread, double a, double b, double h,
double eps) {

    while(!start_flag.load(std::memory_order_relaxed)){}

    double pi_4 = M_PI / 4.0;

    for(long long iter = 0; iter < iters_for_this_thread; ++iter) {
        for(double x = a; x <= b; x += h) {
            double y_val = exp(x * cos(pi_4)) * cos(x * sin(pi_4));
            double s_val = 0.0;
            double current_term = 0.0;
            int k = 0;

            do {

                double numerator = asm_mul(cos(k * pi_4), pow(x, k));
                double denominator = asm_factorial(k);
                current_term = numerator / denominator;
                s_val = asm_add(s_val, current_term);
                ++k;

            } while(abs(y_val - s_val) >= eps && k < 100);
        }

        global_completed_iters.fetch_add(1, std::memory_order_relaxed);
    }
}

int main() {

    double a = 0.1, b = 1.0, h = 0.1, eps = 0.0001;
    long long max_iterations = 10000;

    unsigned int num_threads = std::thread::hardware_concurrency();

    if(num_threads == 0) {
        num_threads = 16;
    }

    cout << "Lab 2:\nDefault values:" << endl;

```

```

cout << "a: " << a << endl;
cout << "b: " << b << endl;
cout << "h: " << h << endl;
cout << "eps: " << eps << endl;
cout << "max_iterations: " << max_iterations << endl;
cout << "num_threads: " << num_threads << endl;

ofstream csv("data.csv");
csv << "Time,Iterations\n";

cout << "Starting CPU test (SMT ON/OFF)..." << endl;

global_completed_iters = 0;

long long iters_per_thread = max_iterations / num_threads;

vector<std::thread> threads;

for(unsigned int i = 0; i < num_threads; ++i) {
    threads.push_back(std::thread(worker_task, iters_per_thread, a, b, h,
eps));
}

std::this_thread::sleep_for(std::chrono::milliseconds(1));

auto start_time = chrono::high_resolution_clock::now();

start_flag.store(true, std::memory_order_relaxed);

long long record_step = max_iterations / 100;
if(record_step == 0) {
    record_step = 1;
}

long long next_target = record_step;

while(true) {
    long long current_iters =
global_completed_iters.load(std::memory_order_relaxed);
    if(current_iters >= next_target || current_iters >= max_iterations) {
        auto current_time = chrono::high_resolution_clock::now();
        chrono::duration<double> elapsed = current_time - start_time;
        csv << elapsed.count() << "," << current_iters << "\n";
        next_target = current_iters + record_step;
    }
    if(current_iters >= (num_threads * iters_per_thread)) {
        break;
    }
}

for(auto& t : threads) {
    t.join();
}

```

```
csv.close();  
  
cout << "Succesfull create data.csv" << endl;  
  
return 0;  
}
```

Данная лабораторная была выполнена на ОС Kali Linux, поэтому все команды будут относиться именно к этому дистрибутиву.

Перед запуском программы ее нужно было скомпилировать, поэтому я использовал команду `g++ -O0 main.cpp -o lab3 -lOpenCL`, где `-O0` указывает не оптимизировать исходный код, т.к. это может сказаться на результате.

Для получения достоверных метрик производительности и полного исключения влияния графического сервера, параметры загрузки ядра Linux (GRUB) были модифицированы: добавлен параметр `3`, что обеспечило загрузку системы в строгом текстовом режиме (`multi-user.target`).

Сначала запустим программу с 10 000 итераций, распределив их на 100 потоков GPU, для сравнения работы GPU на незначительном и значительном количестве итераций и сравним результаты. Перед запуском программы пришлось использовать команду `export RUSTICL_ENABLE=radeonsi` в терминале, т.к. без нее программа не могла обнаружить GPU, возможно это связано с тем, что в моем компьютере нет дискретной GPU, а используется встроенная GPU.

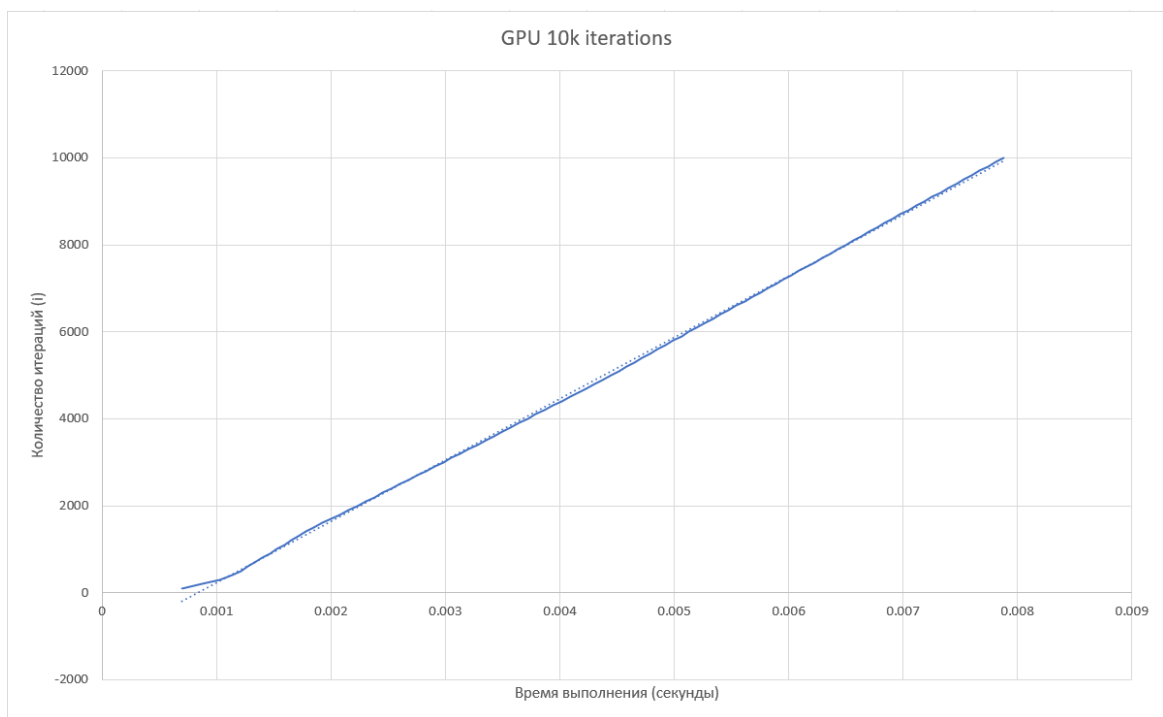


Рисунок 2.1 – График выполнения 10 000 итераций на GPU

Далее запустим программу с 1 000 000 итераций, распределив их между 10 000 потоками GPU.

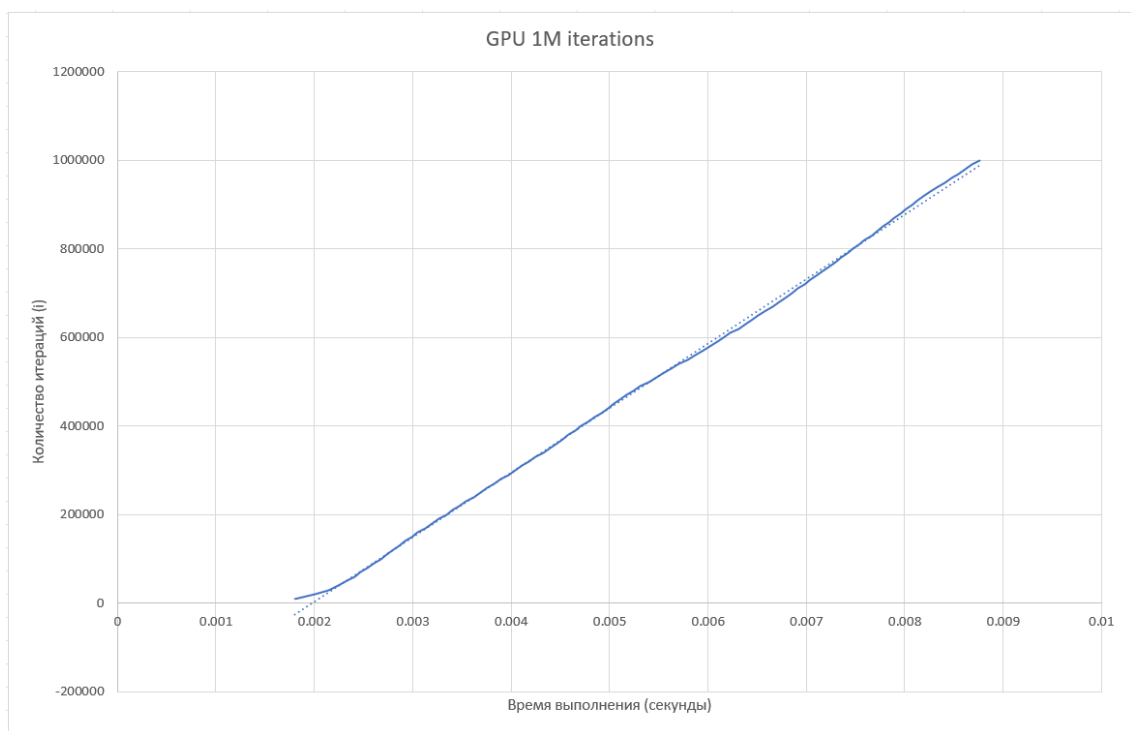


Рисунок 2.2 – График выполнения 1 000 000 итераций на GPU

ВЫВОД

В ходе практической работы был реализован алгоритм распараллеливания математической задачи на встроенном графическом процессоре (AMD iGPU) с использованием открытого стандарта OpenCL.

На графиках производительности наглядно видно радикальное преимущество графического ускорителя при решении массивно–параллельных задач. В то время как многоядерный центральный процессор (CPU) тратил значительное время на выполнение значительного количества итераций, интегрированная видеокарта (iGPU) обработала 1 000 000 итераций за менее 9 мс. Несмотря на то, что запуск 10 000 итераций с 100 потоками показал достаточно посредственные результаты, при этом при запуске программы на 1 000 000 итераций с 10 000 потоками мы получили, что первые 10 000 итераций выполнялись за 1.8 мс, что является лучшим результатом за все лабораторные.

Столь высокая производительность обусловлена разницей архитектур. Центральный процессор (CPU) обладает небольшим числом сложных ядер, ориентированных на минимизацию задержек и быстрое переключение контекста.

Графический процессор AMD спроектирован по парадигме SIMT (Single Instruction, Multiple Threads). В его состав входят потоковые процессоры, сгруппированные в вычислительные блоки. Они не имеют сложных блоков предсказания ветвлений, но способны одновременно применять одну и ту же математическую инструкцию тысячам независимых потоков данных (в нашем случае – к независимым итерациям математического ряда).

Стандарт OpenCL позволил абстрагироваться от ручного написания аппаратного ассемблера (в отличие от Практической работы №1 и №2). Математическое ядро (Kernel) компилировалось JIT-компилятором драйвера AMD непосредственно в низкоуровневые инструкции GPU, обеспечив максимальную пропускную способность вычислительной системы.